

26.6.11 Meeting 12

一. M11反思 为什么三个 prompt 一起反而变差？

每个节点最终 prompt：

$$P_i = \{p_d, p_s, p_o\}$$

然后做融合。

但是：

现在用的是：

```
no_attention=True
```

即平均融合。

那么：

一个节点可能同时满足：

```
degree_pt  
sim_pt  
ood_pt
```

最终：

$$p = \frac{p_d + p_s + p_o}{3}$$

但论文原意是：

不同 prompt 处理不同攻击模式。

平均以后：

互相抵消

甚至：

正常节点被加多个prompt

导致过平滑。

所以出现：

Single Prompt > Multi Prompt

二. Filter 代码逻辑

模块	similarity 来源	判断对象	结果
<code>sim_pt</code>	原始特征 <code>x</code>	节点的平均邻居相似度	给节点加 <code>sim_pt</code>
<code>out_detect_pt</code>	原始特征 <code>x</code>	每条边	给可疑边两端节点加 <code>out_detect_pt</code>

模块	similarity 来源	判断对象	结果
<code>τ_tune</code>	第一次 GNN 后的 node embedding	每条边	删除低相似度边
<code>degree_pt</code>	不用 similarity, 用 degree	节点度数	给低度数节点加 <code>degree_pt</code>

申明：

similarity 在这里被用了两次，但不是同一个 similarity，也不是同一个阶段。

`sim_pt / ood:`

用原始/加 prompt 前的节点特征 x 算 similarity
目的：选节点加 prompt
不删边

`τ_tune:`

用第一次 GNN 后的 node embedding 算 similarity
目的：删边

RobustPrompt_T.py

sim_pt：节点选择用 similarity

位置：RobustPrompt_T.py

它先算每条边两端原始特征 `x` 的 cosine：

```
e = torch.sum(x_norm[edge_index[0]] * x_norm[edge_index[1]],
dim=1)
```

然后按目标节点聚合邻居相似度：

```
c = c.scatter_add_(dim=0, index=col.unsqueeze(1), src=e)
deg = degree(col, x.size(0), dtype=x.dtype)
csim = c / deg
```

最后选出平均邻居相似度低的节点：

```
node_use_sim_pt = torch.nonzero(csim <= self.pt_dict['sim_p
t'])
```

所以 `sim_pt` 的逻辑是：

```
if 节点和邻居的平均 similarity <= pt_sim_threshold:
    给这个节点加 sim_pt
```

不删边

ood：边异常检测用 similarity，但结果还是选节点

位置：[RobustPrompt_T.py](#)

它也用原始特征 `x` 算边两端 cosine：

```
e = torch.sum(x_norm[edge_index[0]] * x_norm[edge_index[1]],
dim=1)
ood_edge_mask = e <= ood_threshold
ood_edges = edge_index[:, ood_edge_mask]
node_use_ood_pt = torch.unique(torch.cat([ood_edges[0], ood_e
dges[1]]))
```

所以 `ood` 的逻辑是：

```
if 边两端 similarity <= pt_out_detect_threshold:
    认为这条边可疑
    给这条边两端节点加 out_detect_pt
```

它虽然算了 `ood_edge_mask`，但这个 mask 没有用于 `edge_index[:, keep_mask]`，所以不删边。

τ _tune：删边用 similarity

位置：[RobustPrompt_T.py](#)

先加 prompt：

```
g_mutiftpt, node_use_each_pt_whole_graph = self.add_muti_pt(g  
graph, device)
```

然后第一次 GNN：

```
node_emb_intermediate = gnn(g_mutiftpt.x, g_mutiftpt.edge_ind  
ex)
```

再用中间 embedding 算边两端 similarity：

```
edge_sim_intermediate = F.cosine_similarity(  
    node_emb_intermediate[g_mutiftpt.edge_index[0]],  
    node_emb_intermediate[g_mutiftpt.edge_index[1]],  
    dim=1  
)
```

最后真正剪边：

```
tune_keep_mask = edge_sim_intermediate >= self.pt_threshold  
tuned_edge_index = g_mutiftpt.edge_index[:, tune_keep_mask]
```

所以 τ_{tune} 是：

```
if GNN中间embedding的边 similarity < pt_threshold:  
    删除这条边
```

RobustPrompt_I

`RobustPrompt_I.py` 和 `RobustPrompt_T.py` 不一样：

RobustPrompt-T:

sim/dt/ood 选节点加 prompt

τ_{tune} 用 GNN 中间 embedding 再剪边

RobustPrompt-I:

sim/dt 选节点加 prompt

ood 目前没实现

然后直接用 filter_module 对加 prompt 后的图剪边

真正删边：filter_module

位置：[RobustPrompt_I.py](#)

```
g.x = self.add_pt(g.x, g_mutiftpt_final_output)
filter_output = self.filter_module(g)
pruned_edge_index = g.edge_index[:, filter_output['edge_mask']]
pruned_g_after_pt = Data(x=g.x, edge_index=pruned_edge_index,
y=g.y)
```

这里才是真正剪边。流程是：

原始子图

- > 根据 sim_pt / degree_pt / other_pt 选节点
- > 给节点加 prompt，得到新的 g.x
- > 把加 prompt 后的图传入 filter_module
- > filter_module 算 edge_mask
- > edge_index[:, edge_mask] 保留边
- > 后续 GNN 用剪枝后的图

1. 先给节点特征加 prompt：`g.x = self.add_pt(...)`
2. 把加完 prompt 的图传给 filter：`filter_output = self.filter_module(g)`
3. 取出 `edge_mask`
4. 用 `g.edge_index[:, edge_mask]` 保留可信边
5. 构造新图 `pruned_g_after_pt`
6. 后面的 GNN 用剪枝后的 `prompted_graph.edge_index`

GPPT 里怎么用

```
filter_output = self.filter_module(filter_graph)
edge_mask = filter_output.get('edge_mask', None)

if edge_mask is not None:
    return edge_index[:, edge_mask]
```

这个函数在两个地方用：

- 初始化 token 时：[GPPTPrompt.py](#)
- forward 推理/训练时：[GPPTPrompt.py](#)

效果是：GPPT 的 `SimpleMeanConv()` 聚合邻居特征时，使用的是 filter 后的边。

三. Transductive VS Inductive

GPromptShield 其实是：

```
Step1:
攻击整张图

Step2:
从攻击后的图提取子图

Step3:
做Inductive Prompt Learning
```

我们的结果跟论文差距大有可能的原因是：

MetaAttack 最擅长破坏的是：

Transductive Message Passing

当把节点切成大量局部子图后：

攻击影响会被稀释

四.跑inductive实验（Inductive MD-PT）

一、代码修改

从 RobustPrompt-T 的全图近似实现 向论文中 RobustPrompt-I 的 Inductive MD-PT（Multi-Defense Prompt Tuning）框架 的进一步靠拢。

1. RobustPrompt-I Prompt机制升级

原实现仅保留：

```
{'other_pt': 'all'}
```

相当于所有节点共享一个统一 Prompt。

现改为论文中的 Multi-Defense Prompt：

```
{  
    'sim_pt': ...,  
    'degree_pt': ...,  
    'out_detect_pt': ...,  
    'other_pt': 'all',  
}
```

分别对应：

- Similarity Prompt
- Degree Prompt
- Outlier Detection Prompt
- Default Prompt

从而实现论文中的 MD-PT 设计。

2. 超参数开放为命令行配置

原先部分参数写死在代码中。

目前已支持：

```
p_plus  
use_attention  
pt_threshold  
temperature  
  
weight_mse  
weight_kl  
weight_constraint  
  
cosine_constraint  
  
prompt_lr
```

便于后续系统调参。

3. Eval阶段修改为 Add-only

修改了：

```
RobustPrompt_I.forward()
```

推理阶段现在只执行：

```
Add Prompt
```

不执行：

```
 $\tau_{\text{tune}}$  Edge Pruning
```

与论文设定保持一致。

即：

```
Train:  
Prompt +  $\tau_{\text{tune}}$   
  
Eval:  
Prompt only
```

4. 实现训练阶段 τ_{tune}

训练流程目前变为：

```
Add Multi-Defense Prompt  
↓  
GNN Forward #1  
↓  
获得节点Embedding  
↓  
基于Embedding Cosine Similarity  
执行 $\tau_{\text{tune}}$ 剪边  
↓  
GNN Forward #2  
↓  
Graph Embedding  
↓  
CE + MSE + KL + Constraint Loss
```

相比之前版本，多出了：

```
Prompt → Edge Tuning → Re-encode
```

步骤。

为什么做

因为这是论文里 GPromptShield / MD-PT 的关键训练机制： τ_{tune} 不是普通预处理 filter，而是训练阶段的间接增强策略。

它的作用是：

1. 先让 prompt 影响节点表示；
2. 再基于 prompt 后的 embedding 判断哪些边不可靠；
3. 训练时删掉低相似边，让模型不要依赖可疑边；
4. 迫使 prompt 学出更鲁棒的表示。

所以相比旧版 RobustPrompt-l.forward() 里直接 filter_module(g) 剪边，现在更贴近论文：

旧版：加 prompt 后直接用规则 filter 剪边
新版：加 prompt -> GNN -> embedding cosine τ_{tune} -> 再 GNN

同时 eval 阶段没有剪枝，因为论文设定里 τ_{tune} 是训练策略，不是推理策略

5. 补充OOD Prompt原型实现

目前实现的是简化版本：

```
Edge两端原始特征Cosine Similarity
↓
判断OOD Edge
↓
给对应节点添加OOD Prompt
```

暂未实现论文中的完整OOD Detector。

因此目前属于：

```
Feature-Cosine Proxy
```

版本。

6. 数值稳定性修改

增加：

```
+1e-12
```

避免归一化除零。

同时：

```
softmax(dim=-1)  
log_softmax(dim=-1)
```

显式指定维度。

并增加：

```
Gradient Clipping
```

用于：

- Prompt参数
- Answering Head

训练稳定化。

二、实验设计

本轮实验采用单Filter Ablation。

即每次只启用一个Prompt Filter，其余Filter关闭。

Similarity Prompt

```
--pt_sim_threshold ${sim_t}  
  
--pt_degree_threshold -1  
--pt_out_detect_threshold -1
```

扫描：

```
0.2
0.3
0.4
0.5
0.6
```

Degree Prompt

```
--pt_sim_threshold -1

--pt_degree_threshold ${deg_t}

--pt_out_detect_threshold -1
```

扫描：

```
1
2
3
5
```

OOD Prompt

```
--pt_sim_threshold -1

--pt_degree_threshold -1

--pt_out_detect_threshold ${ood_t}
```

扫描：

```
0.3
0.4
0.5
```

```
0.6  
0.7
```

所有实验固定：

```
prompt_lr=0.004  
  
pt_threshold=0.25  
  
weight_mse=0.1  
  
weight_kl=0.1  
  
weight_constraint=0.0  
  
no_attention
```

每组：

```
5 Seeds  
6 Pollution Levels
```

共计：

```
84 Logs
```

三、实验结果

整体观察

虽然部分配置取得了较高Accuracy，但部分实验出现：

```
Loss nan
```

导致许多Seed无法完成训练。

因此目前结果主要反映：

训练稳定性问题

而不是：

最佳Prompt阈值

表面最优结果

Ptb Rate	Best Filter	Threshold	Acc
0.00	Degree	5	0.593
0.05	OOD	0.4	0.444
0.10	OOD	0.5	0.198
0.15	Similarity	0.4	0.171
0.20	Similarity	0.4	0.103
0.25	Degree	5	0.067

但由于存在Loss Nan情况，需要后续做更多调试

四、问题分析

目前最大的瓶颈并不是Prompt阈值选择，而是训练稳定性。

原因1：超参数小子图场景下梯度波动明显更大

当前使用：

```
prompt_lr=0.004
```

```
weight_mse=0.1
```

```
weight_kl=0.1
```

需要更谨慎地再次调节

原因2：KL损失方差过大

当前KL计算：

Prompt节点Embedding均值

vs

Non-Prompt节点Embedding均值

在小子图中：

Prompt节点数量可能极少

导致KL统计极不稳定。

```
weight_kl=0.1
```

是当前最可疑的NaN来源之一。

原因3： τ_{tune} 进一步稀疏化图结构

训练过程：

```
Prompt  
→ GNN  
→  $\tau_{\text{tune}}$ 剪边  
→ GNN
```

对于部分子图：

```
pt_threshold=0.25
```

可能剪除过多边。

导致：

```
Embedding方差增大
```

进一步放大：

```
MSE  
KL
```

损失项。

原因4：训练过程缺少Best Checkpoint恢复

目前流程：

```
Loss NaN  
↓  
Early Stop  
↓  
Seed记为NaN
```

即使：

```
Epoch 180性能很好  
Epoch 195出现NaN
```

最终整个Seed仍被判定失败。

因此统计结果进一步恶化。

五、下一步计划

我认为下一阶段应优先解决训练稳定性，然后继续扫描Filter阈值。

渐进式排查

Step 1：CE-only稳定性测试

```
prompt_lr=0.001

pt_threshold=-1.1

weight_mse=0

weight_kl=0

weight_constraint=0
```

验证主路径是否稳定。

Step 2：只开启 τ _tune

```
prompt_lr=0.001

pt_threshold=0.25

weight_mse=0

weight_kl=0
```

验证剪边机制是否导致NaN。

Step 3：加入小MSE

```
weight_mse=0.01

weight_kl=0
```

观察稳定性变化。

Step 4：加入极小KL

```
weight_mse=0.01
```

```
weight_kl=0.001
```

验证KL对训练稳定性的影响。

Step 5：重新进行Filter扫描

只有在：

5/5 Seeds稳定完成训练

之后，再重新扫描：

```
Similarity Prompt  
Degree Prompt  
OOD Prompt
```